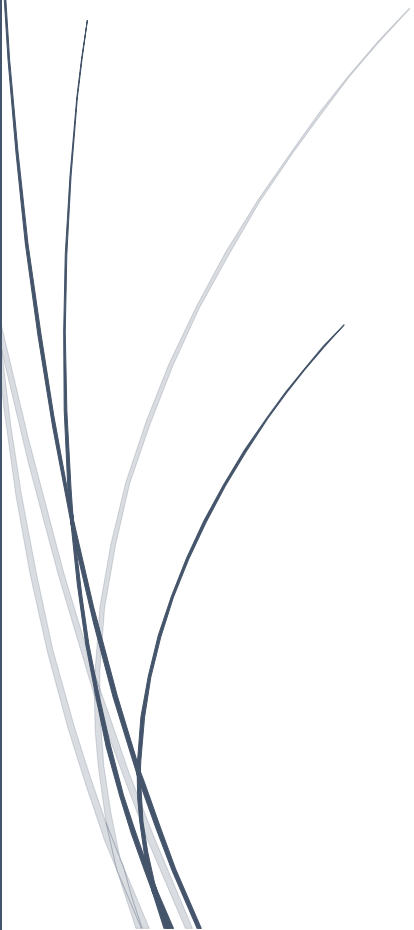




PHARMATAALKDESK



Mennea Antonio

Sommario

Cos'è PharmaTalkDesk?	3
Cosa è stato utilizzato?	5
Come posso avviare la web app?	7

Cos'è PharmaTalkDesk?

PharmaTalkDesk è un'applicazione web interattiva progettata per fornire informazioni complete sui medicinali attraverso un'interfaccia moderna e intuitiva. Il sistema combina la visualizzazione di dati strutturati con l'intelligenza artificiale conversazionale per offrire un'esperienza utente completa e professionale.

L'applicazione supporta completamente due lingue:

- **Italiano:** Lingua principale con traduzioni complete di tutti i contenuti;
- **Inglese:** Interfaccia alternativa per utenti internazionali;

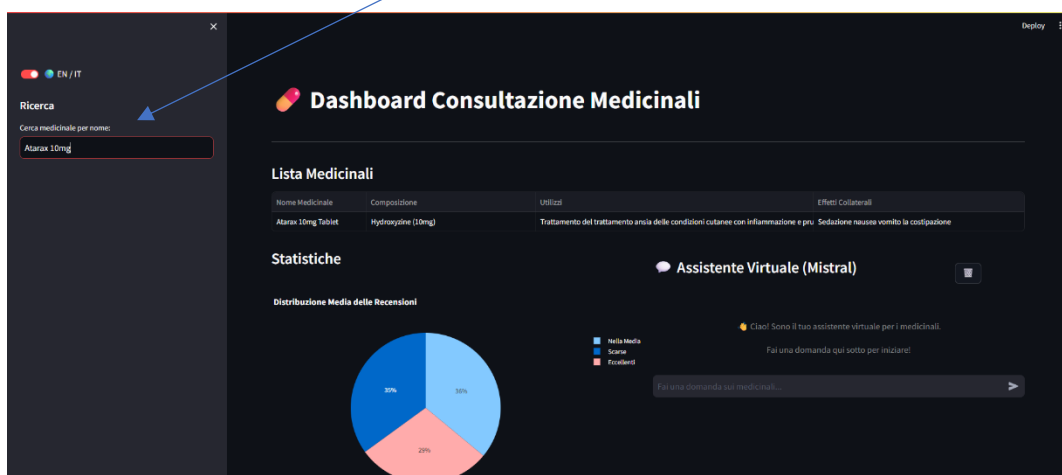
Infatti è possibile selezionare la lingua che si desidera usare durante l'esecuzione della web app, il tutto tramite un toggle dinamico.



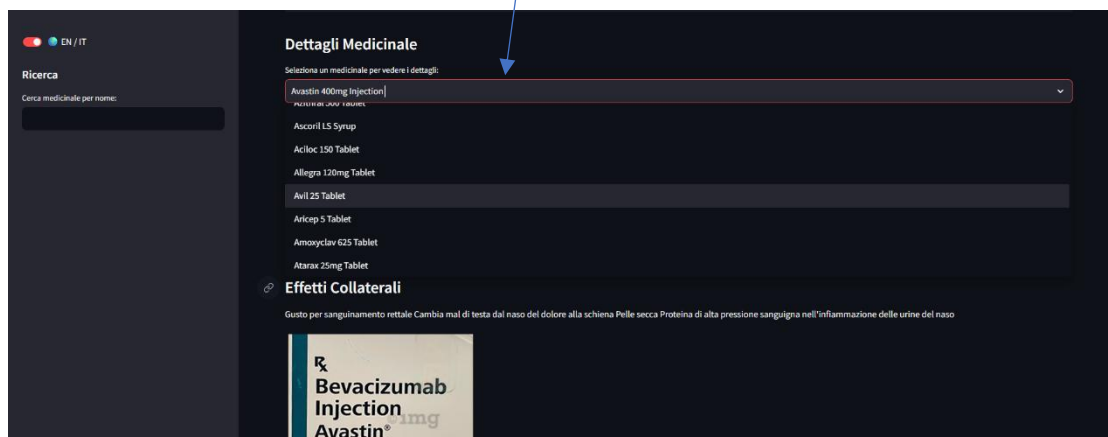
Il sistema include un traduttore automatico integrato che converte dinamicamente le informazioni sui medicinali dall'inglese all'italiano, garantendo contenuti sempre aggiornati e accurati.

Gli utenti possono:

- Effettuare ricerche rapide per nome del medicinale;

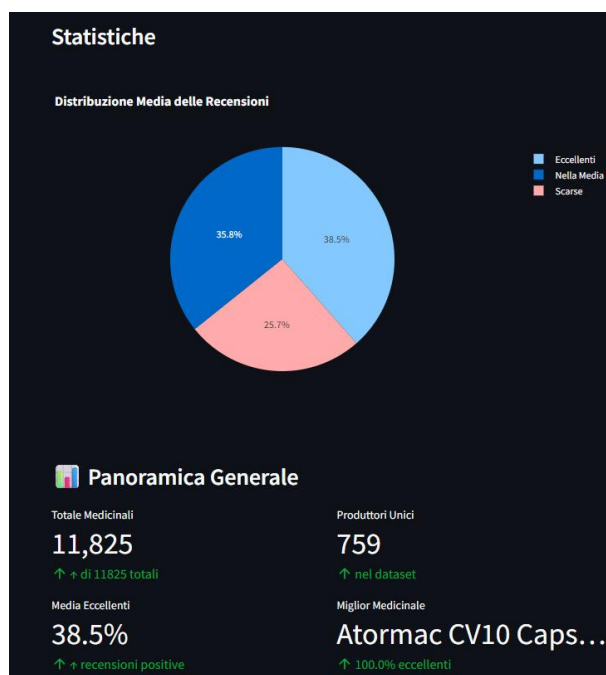


- Accedere a informazioni dettagliate con un singolo clic;



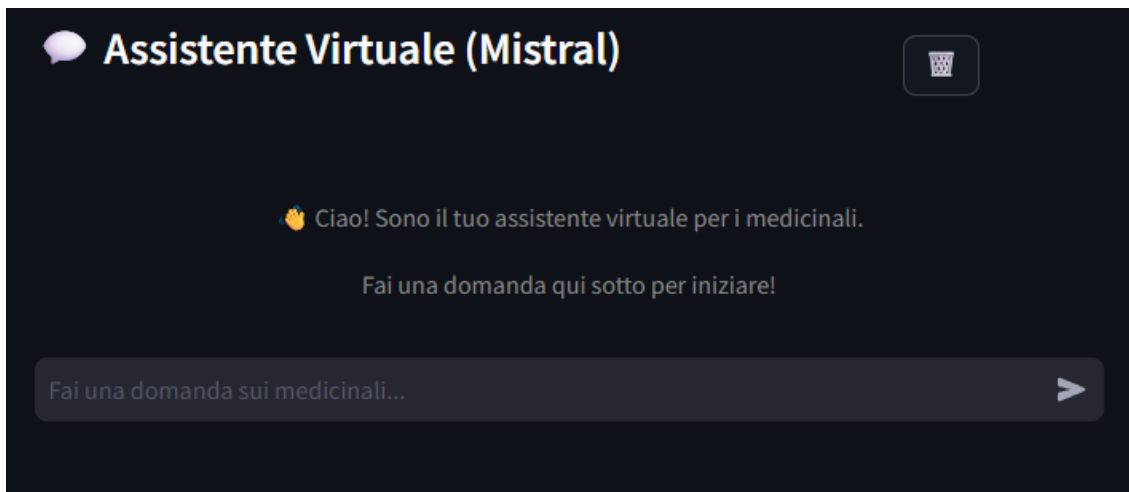
Il sistema presenta:

- **Grafici interattivi** che mostrano la distribuzione delle recensioni
- **Metriche aggregate** sul database (numero totale medicinali, produttori unici)
- **Indicatori di performance** per identificare i medicinali meglio recensiti
- **Panoramiche generali** con statistiche riassuntive



L'applicazione integra un chatbot intelligente alimentato dal modello **Mistral** che:

- Risponde a domande specifiche sui medicinali in linguaggio naturale;
- Fornisce informazioni contestualizzate basate sui dati del database;
- Mantiene conversazioni professionali mantenendo un approccio medico responsabile;
- Utilizza un sistema di cache per ottimizzare le risposte;
- Enfatizza sempre l'importanza della consultazione medica professionale.



Cosa è stato utilizzato?

Per realizzare questa Web App sono state utilizzate le seguenti dipendenze:

- Streamlit: Framework per la creazione rapida di applicazioni web interattive. Permette di trasformare script Python in dashboard web complete senza necessità di conoscenze HTML/CSS/JavaScript. Nel progetto gestisce l'intera interfaccia utente, i widget interattivi, e il layout della dashboard.
- Pandas: Libreria fondamentale per la manipolazione e analisi dei dati, utilizzata per:
 - Caricare e gestire il dataset dei medicinali in formato CSV;
 - Eseguire operazioni di ricerca e aggregazione dei dati;
 - Preparare i dati per la visualizzazione;
- Plotly: Libreria per la creazione di grafici interattivi e visualizzazioni avanzate. Nel progetto genera:
 - Grafici a torta per la distribuzione delle recensioni;
 - Visualizzazioni statistiche interattive;
 - Chart responsive che si adattano all'interfaccia web;
- Deep-translator: Libreria per la traduzione automatica del testo utilizzando vari servizi di traduzione. Usata principalmente per tradurre il dataset al primo avvio dall'inglese alla lingua italiana;

- **Fastapi:** Framework moderno per la creazione di API REST ad alte performance.
Nel progetto:
 - Gestisce l'endpoint per il chatbot AI;
 - Fornisce un'interfaccia API per le richieste del sistema di chat;
 - Garantisce comunicazione asincrona tra frontend e backend;
- **Uvicorn:** Server ASGI (Asynchronous Server Gateway Interface) ad alte performance per applicazioni Python asincrone. Utilizzato per:
 - Eseguire il server FastAPI in background;
 - Gestire le connessioni HTTP in modo asincrono;
 - Fornire un runtime stabile per l'API del chatbot;
- **Requests:** Libreria per effettuare richieste HTTP in modo semplice ed elegante.
Nel progetto serve per:
 - Comunicare con il servizio Ollama (modello AI Mistral);
 - Inviare richieste al server locale per il chatbot;
 - Verificare lo stato dei servizi esterni;
- **Nest-asyncio:** Libreria che permette l'esecuzione di loop asincroni annidati.
Necessaria per:
 - Risolvere conflitti tra i loop asincroni di Streamlit e FastAPI;
 - Permettere l'esecuzione simultanea di operazioni asincrone;
 - Garantire compatibilità tra diversi framework asincroni;
- **Pydantic:** Libreria per la validazione dei dati e la gestione dei modelli usando le type hints di Python. Utilizzata per:
 - Definire la struttura delle richieste API del chatbot;
 - Validare automaticamente i dati in input;
 - Garantire type safety nelle comunicazioni API;

Queste librerie lavorano insieme per creare un ecosistema completo:

- **Streamlit + Pandas + Plotly:** Gestiscono l'interfaccia utente e la visualizzazione dei dati
- **FastAPI + Uvicorn + Pydantic:** Forniscono il backend per il chatbot AI
- **Requests:** Collega il frontend con i servizi AI esterni
- **Deep-translator:** Abilita il supporto multilingua
- **Nest-asyncio:** Garantisce la compatibilità asincrona tra tutti i componenti

Questa combinazione di tecnologie permette di avere un'applicazione moderna, performante e scalabile con funzionalità avanzate di AI e supporto internazionale.

Oltre a tutte le dipendenze usate, per far funzionare alla perfezione la nostra web app bisogna installare Ollama. [Ollama](#) è un piccolo “centro di comando” per i modelli di intelligenza artificiale che sfrutta i componenti del proprio PC per funzionare, più performante è prima si riceve la risposta da parte dell’LLM.

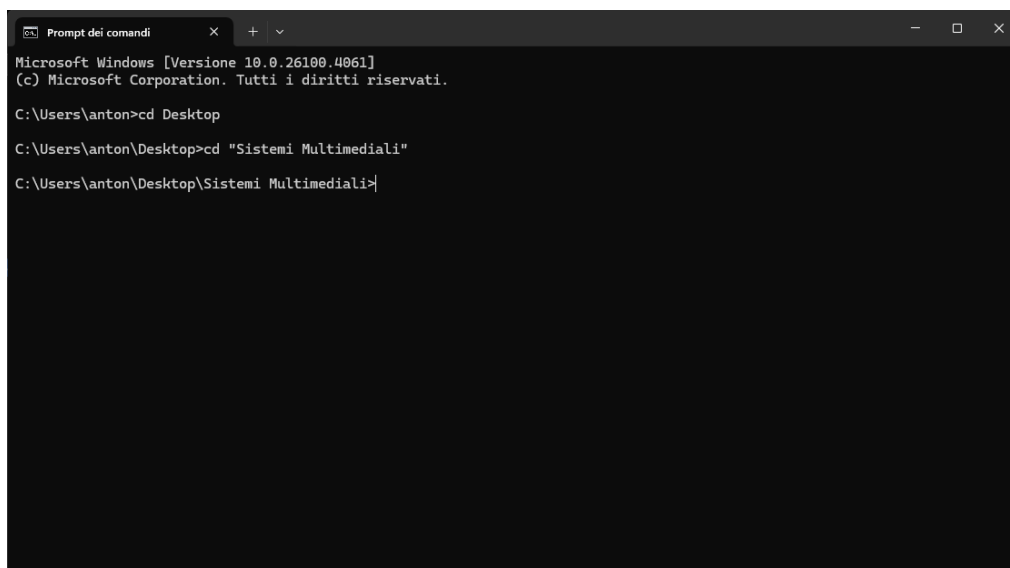
Una volta installato Ollama, esso è un guscio vuoto, per riempirlo bisogna identificare l’LLM che più si adatta alle nostre caratteristiche. Ollama fornisce una [lista](#) di tutti gli LLM che possono essere utilizzati, bisogna prestare attenzione nella selezione dell’LLM da utilizzare, ci sono dei requisiti per scaricare ed utilizzare l’LLM scelto, se viene scelto uno in cui il suo peso supera la capienza di memoria disponibile verrà mostrato un errore e tale LLM non potrà essere utilizzato, più è grosso l’LLM e più tempo ci metterà la macchina nell’elaborare una possibile risposta alla domanda dell’utente.

È possibile installare Ollama tramite il suo installer, invece per quanto riguarda l’LLM da scaricare bisogna aspettare che Ollama concluda la sua installazione e successivamente è possibile eseguire il comando che è presente accanto al nome dell’LLM che si desidera scaricare ed usare, nel nostro caso una volta terminata l’installazione di Ollama apriamo una shell ed eseguire il comando:

- ollama run mistral

Come posso avviare la web app?

Per prima cosa scarichiamo la repository del progetto caricato su [GitHub](#), una volta scaricato lo zip dobbiamo estrarre tutto il suo contenuto, una volta estratto apriamo una shell che punta a questa cartella:



```
Prompt dei comandi
Microsoft Windows [Versione 10.0.26100.4061]
(c) Microsoft Corporation. Tutti i diritti riservati.

C:\Users\anton>cd Desktop
C:\Users\anton\Desktop>cd "Sistemi Multimediali"
C:\Users\anton\Desktop\Sistemi Multimediali>
```

Ora possiamo installare Python per avviare la web app, bisogna prestare attenzione alla versione di Python che viene installata, per eseguire la web app è necessaria la versione “Python 3.11.x”, se esso non è installato è possibile avviare il prompt di comando(CMD) ed eseguire il seguente comando:

- Per Windows:
 - `winget install --id Python.Python3.11 -e`

Questo comando installerà l’ultima versione disponibile di python in versione 3.11;

- Per Linux:
 - `sudo apt update;`
 - `sudo apt install -y python3.11`
- Per MacOS:
 - Con Homebrew:
 - `brew update`
 - `brew install python@3.11`
 - `brew link --force --overwrite python@3.11`
 - Con installer ufficiale (.pkg):
 - `curl -O https://www.python.org/ftp/python/3.11.6/python-3.11.6-macos11.pkg`
 - `sudo installer -pkg python-3.11.6-macos11.pkg -target /`

Una volta installato Python all’interno della macchina bisogna installare le dipendenze, visto che tutte le dipendenze necessarie per far funzionare la web app sono state inserite all’interno del file “requirements.txt”, possiamo aprire una shell ed eseguire questo comando:

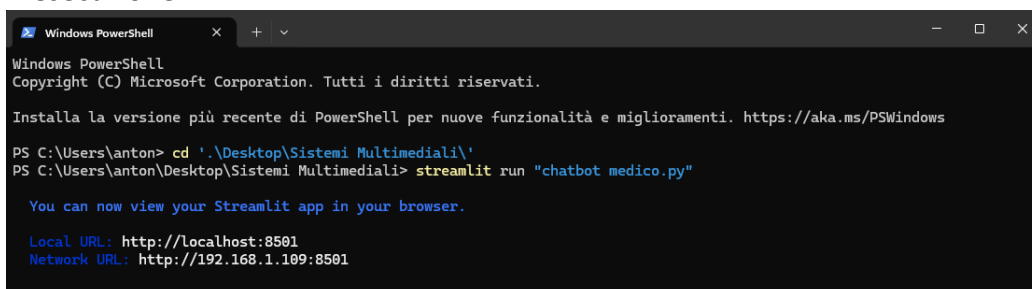
- `pip install -r requirements.txt`

Grazie a questo comando installeremo tutte le dipendenze in un unico comando.

Ora che abbiamo tutto ciò che è necessario per eseguire la web app, andiamo a riprendere la shell che punta alla cartella in cui è presente la nostra web app e digitiamo il seguente comando:

- `streamlit run "chatbot medico.py"`

Una volta eseguito il comando, nella shell verranno visualizzati come output dei “link” che servono per accedere alla web app qualora essa viene chiusa ma la shell in cui è stato eseguito il comando è ancora in esecuzione.



```

Windows PowerShell
Copyright (C) Microsoft Corporation. Tutti i diritti riservati.

Installa la versione più recente di PowerShell per nuove funzionalità e miglioramenti. https://aka.ms/PSWindows

PS C:\Users\anton> cd '.\Desktop\Sistemi Multimediali\'
PS C:\Users\anton\Desktop\Sistemi Multimediali> streamlit run "chatbot medico.py"

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.109:8501
  
```